UNIVERSITY
OF COLOGNE

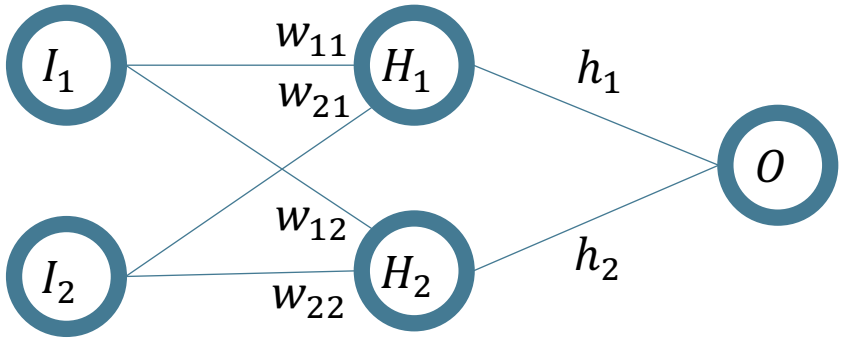
WORKSHOP 05

Advanced Analytics and Applications

Agenda

1	**Lecture Recap**
2	Mid Term Quiz
3	Coding Tasks

Backprop Calculation Example



Assume we have the following neural network setup.

- Learning rate $\eta = 0.05$
- Linear activation function
- One Data Point:

  $I_1 = 2 \quad I_2 = 3 \quad O = 1$
- Random Initial weights:

  $w_{11} = 0.11 \quad h_1 = 0.14$

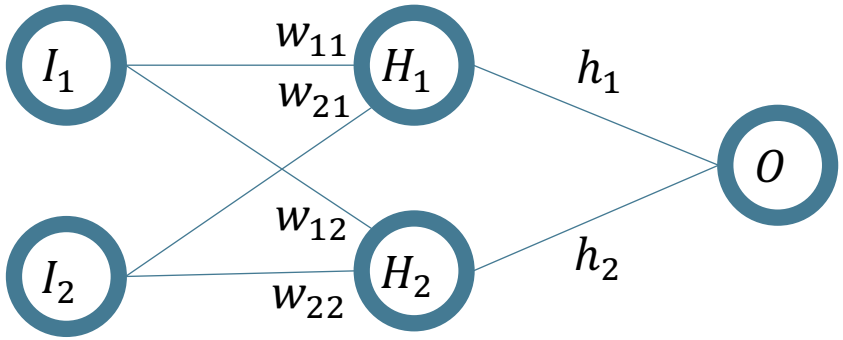
  $w_{21} = 0.21 \quad h_2 = 0.15$

  $w_{12} = 0.12$

  $w_{22} = 0.08$

Backprop Calculation Example

- Question 2: Calculation BackProp



$$\hat{O} = [2 \quad 3] * \begin{bmatrix} 0.11 & 0.12 \\ 0.21 & 0.08 \end{bmatrix} * \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} = 0.191$$

a) Calculate the prediction using this configuration.

- Learning rate $\eta = 0.05$
- Linear activation function
- One Data Point:

$I_1 = 2 \quad I_2 = 3 \quad O = 1$

- Random Initial weights:

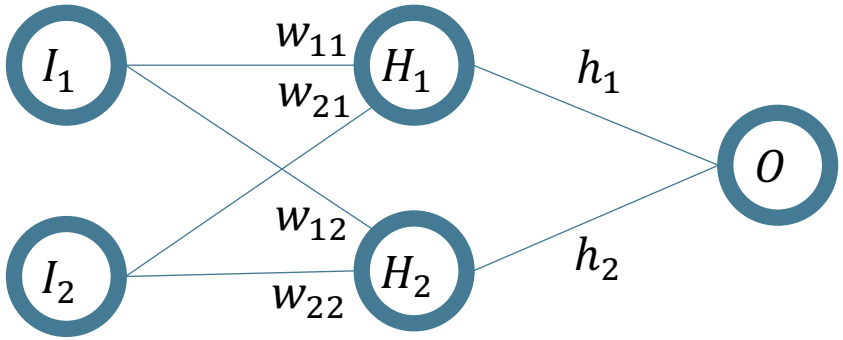
$w_{11} = 0.11 \quad h_1 = 0.14$

$w_{21} = 0.21 \quad h_2 = 0.15$

$w_{12} = 0.12$

$w_{22} = 0.08$

Backprop Calculation Example



L = $\frac{1}{2}$(0.191–1)^2= 0.655

b) Calculate the error using MSE loss function

- Learning rate $\eta = 0.05$
- Linear activation function
- One Data Point:

$I_1 = 2 \quad I_2 = 3 \quad O = 1$

- Random Initial weights:

$w_{11} = 0.11 \quad h_1 = 0.14$

$w_{21} = 0.21 \quad h_2 = 0.15$

$w_{12} = 0.12$

$w_{22} = 0.08$

Backprop Calculation Example

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial \hat{O}} * \frac{\partial \hat{O}}{\partial h_1}$$

$$= \frac{\partial((\hat{O} - O)^2)}{\partial \hat{O}} * \frac{\partial[(I_1 * w_{11} + I_2 w_{21})h_1 + (I_1 * w_{12} + I_2 w_{22})h_2)}{\partial h_1}$$

$$= \frac{\partial((\hat{O} - O))}{\partial \hat{O}} * 2 * (\hat{O} - O) * (I_1 * w_{11} + I_2 w_{21})$$

$$= 2 * (\hat{O} - O) * (I_1 * w_{11} + I_2 w_{21})$$

Similarly for $\frac{\partial L}{\partial h_2} = 2 * (\hat{O} - O) * (I_1 * w_{12} + I_2 w_{22})$

c) Calculate Derivative of Loss with respect to h_1

- Learning rate $\eta = 0.05$
- Linear activation function
- One Data Point:

  $I_1 = 2 \quad I_2 = 3 \quad O = 1$
- Random Initial weights:

  $w_{11} = 0.11 \quad h_1 = 0.14$

  $w_{21} = 0.21 \quad h_2 = 0.15$

  $w_{12} = 0.12$

  $w_{22} = 0.08$

Backprop Calculation Example

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial \hat{O}} * \frac{\partial \hat{O}}{\partial H_1} * \frac{\partial H_1}{\partial w_{11}}$$

$$= \frac{\partial((\hat{O} - O)^2)}{\partial \hat{O}} * \frac{\partial[(H_1)h_1 + (H_2)h_2)]}{\partial H_1} * \frac{\partial H_1}{\partial w_{11}}$$

$(I_1 * w_{11} + I_2 w_{21})$

$$= \frac{\partial((\hat{O} - O))}{\partial \hat{O}} * 2 * (\hat{O} - O) * (h_1) * (I_1)$$

$$= 2 * (\hat{O} - O) * (h_1) * (I_1)$$

Similarly for $\frac{\partial L}{\partial w_{12}} = 2 * (\hat{O} - O) * (h_2) * (I_1)$

$$\frac{\partial L}{\partial w_{21}} = 2 * (\hat{O} - O) * (h_1) * (I_2)$$

$$\frac{\partial L}{\partial w_{22}} = 2 * (\hat{O} - O) * (h_2) * (I_2)$$

c) Calculate Derivative of Loss with respect to w_{11}

- Learning rate $\eta = 0.05$
- Linear activation function
- One Data Point:

  $I_1 = 2 \quad I_2 = 3 \quad O = 1$
- Random Initial weights:

  $w_{11} = 0.11 \quad h_1 = 0.14$

  $w_{21} = 0.21 \quad h_2 = 0.15$

  $w_{12} = 0.12$

  $w_{22} = 0.08$

Backprop Calculation Example

$$w_{11}^{k+1} = w_{11}^{k} - \eta * 2 * (\hat{O} - O) * (h_1) * (I_1)$$

$$w_{12}^{k+1} = w_{12}^{k} - \eta * 2 * (\hat{O} - O) * (h_2) * (I_1)$$

$$w_{21}^{k+1} = w_{21}^{k} - \eta * 2 * (\hat{O} - O) * (h_1) * (I_2)$$

$$w_{22}^{k+1} = w_{22}^{k} - \eta * 2 * (\hat{O} - O) * (h_2) * (I_2)$$

$$h_1^{k+1} = h_1^{k} - \eta * 2 * (\hat{O} - O) * (I_1 * w_{11} + I_2 w_{21})$$

$$h_2^{k+1} = h_2^{k} - \eta * 2 * (\hat{O} - O) * (I_1 * w_{12} + I_2 w_{22})$$

Updating weights using derivates

- Learning rate $\eta = 0.05$
- Linear activation function
- One Data Point:

  $I_1 = 2 \quad I_2 = 3 \quad O = 1$
- Random Initial weights:

  $w_{11} = 0.11 \quad h_1 = 0.14$

  $w_{21} = 0.21 \quad h_2 = 0.15$

  $w_{12} = 0.12$

  $w_{22} = 0.08$

Backprop Calculation Example

$$w_{11}^1 = w_{11}^0 - \eta * 2 * (\hat{0} - 0) * (h_1) * (I_1)$$
$$= 0.11 - 0.05 * 2 * (0.191 - 1) * 0.14 * 2$$
$$\cong 0.13$$

$$w_{12}^1 = 0.14$$

$$w_{21}^1 = 0.23$$

$$w_{22}^1 = 0.10$$

$$h_1^1 = 0.21$$

$$h_2^1 = 0.19$$

d) Updating weights using derivates with real values

- Learning rate $\eta = 0.05$
- Linear activation function
- One Data Point:

  $I_1 = 2 \quad I_2 = 3 \quad O = 1$
- Random Initial weights:

  $w_{11} = 0.11 \quad h_1 = 0.14$

  $w_{21} = 0.21 \quad h_2 = 0.15$

  $w_{12} = 0.12$

  $w_{22} = 0.08$

Backprop Calculation Example

e) Calculate prediction using updated weights

- Learning rate $\eta = 0.05$
- Linear activation function
- One Data Point:

$I_1 = 2 \quad I_2 = 3 \quad O = 1$

- Random Initial weights:

$w_{11} = 0.11 \quad h_1 = 0.14$

$w_{21} = 0.21 \quad h_2 = 0.15$

$w_{12} = 0.12$

$w_{22} = 0.08$

Results in updated prediction:

$$\hat{O} = [2 \quad 3] * \begin{bmatrix} 0.13 & 0.14 \\ 0.23 & 0.10 \end{bmatrix} * \begin{bmatrix} 0.21 \\ 0.17 \end{bmatrix} = 0.2981$$

We got closer to the real O – that‘s how backprop works in a nutshell 🙂

Agenda

1	Lecture Recap
2	**Mid Term Quiz**
3	Coding Tasks

Agenda

1	Lecture Recap
2	Mid Term Quiz
3	**Coding Background and Tasks**

What is Pytorch

- Open source machine learning library
- Developed by Meta's AI Research lab
- It leverages the power of GPUs
- Automatic computation of gradients
- Makes it easier to test and develop new ideas.

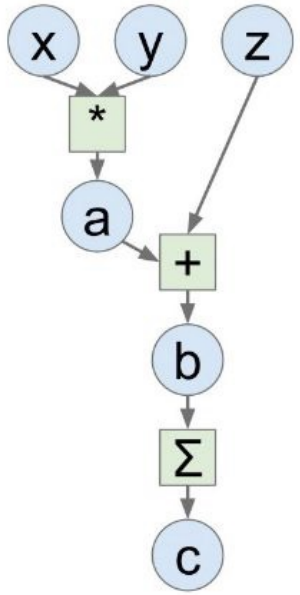
Documentation: https://docs.pytorch.org/docs/stable/index.html

Why Pytorch?

- It is pythonic- concise, close to Python conventions
- Strong GPU support
- Autograd- automatic differentiation
- Many algorithms and components are already implemented
- Similar to NumPy

Very similar to Numpy

Computation Graph



Numpy

```
import numpy as np
np.random.seed(0)

N, D = 3, 4

x = np.random.randn(N, D)
y = np.random.randn(N, D)
z = np.random.randn(N, D)

a = x * y
b = a + z
c = np.sum(b)

grad_c = 1.0
grad_b = grad_c * np.ones((N, D))
grad_a = grad_b.copy()
grad_z = grad_b.copy()
grad_x = grad_a * y
grad_y = grad_a * x
```

Tensorflow

```
import numpy as np
np.random.seed(0)
import tensorflow as tf

N, D = 3, 4

with tf.device('/gpu:0'):
    x = tf.placeholder(tf.float32)
    y = tf.placeholder(tf.float32)
    z = tf.placeholder(tf.float32)

    a = x * y
    b = a + z
    c = tf.reduce_sum(b)

grad_x, grad_y, grad_z = tf.gradients(c, [x, y, z])

with tf.Session() as sess:
    values = {
        x: np.random.randn(N, D),
        y: np.random.randn(N, D),
        z: np.random.randn(N, D),
    }
    out = sess.run([c, grad_x, grad_y, grad_z],
                   feed_dict=values)
    c_val, grad_x_val, grad_y_val, grad_z_val = out
```

PyTorch

```
import torch

N, D = 3, 4

x = torch.rand((N, D),requires_grad=True)
y = torch.rand((N, D),requires_grad=True)
z = torch.rand((N, D),requires_grad=True)

a =x * y
b =a + z
c=torch.sum(b)

c.backward()
```

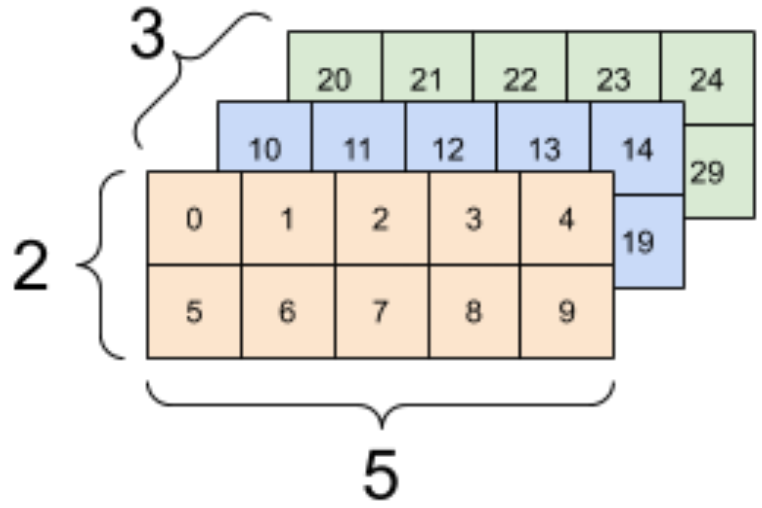
What is tensor?

A **tensor** is a generalization of scalars, vectors, and matrices to higher dimensions. In deep learning and numerical computing, a tensor is simply a **multi-dimensional array**.

Tensors support automatic differentiation (in PyTorch/TensorFlow)

Tensors are similar to NumPy's ndarrays, with the addition being that Tensors can also be used on a GPU to accelerate computing.

Common operations for creation and manipulation of these Tensors are similar to those for ndarrays in NumPy. (rand, ones, zeros, indexing, slicing, reshape, transpose, cross product, matrix product, element wise multiplication)



Agenda

1	Lecture Recap
2	Mid Term Quiz
3	Coding Background and Tasks
4	**Team Assignment**

Examples of things to do ☺

- Make research reproducible by letting people know exactly which data set you are using

Read Data

https://data.cityofchicago.org/Transportation/Taxi-Trips-2024-/ajtu-isnz/data_preview

The data was pre-reduced before downloading in the following ways:

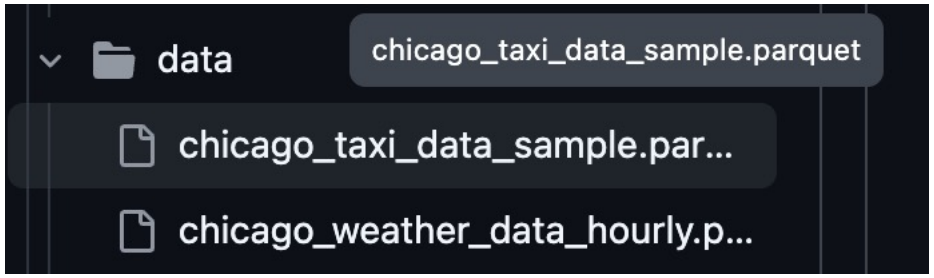
- Trips Seconds 'Not blank'
- Trip Miles 'Not blank' AND 'Not equal 0'
- Pickup Census Tract 'Not blank'
- Dropoff Census Tract 'Not blank'

```
df = pd.read_csv("..\data\local\Taxi_Trips__2024-_prereduced.csv")
```

Our queried dataset can be accessed here: https://drive.google.com/file/d/1-aqjsU7mmQFw[redacted]WXZJSukakJmn/view

Examples of things to do ☺

- Provide small(!) sample data sets so others can run your pipeline and test it out without relying on the full data set being available.
- Sidenote: Using parquet instead of CSV is also advisable, as it is faster to parse and smaller on disk when data is sparse*

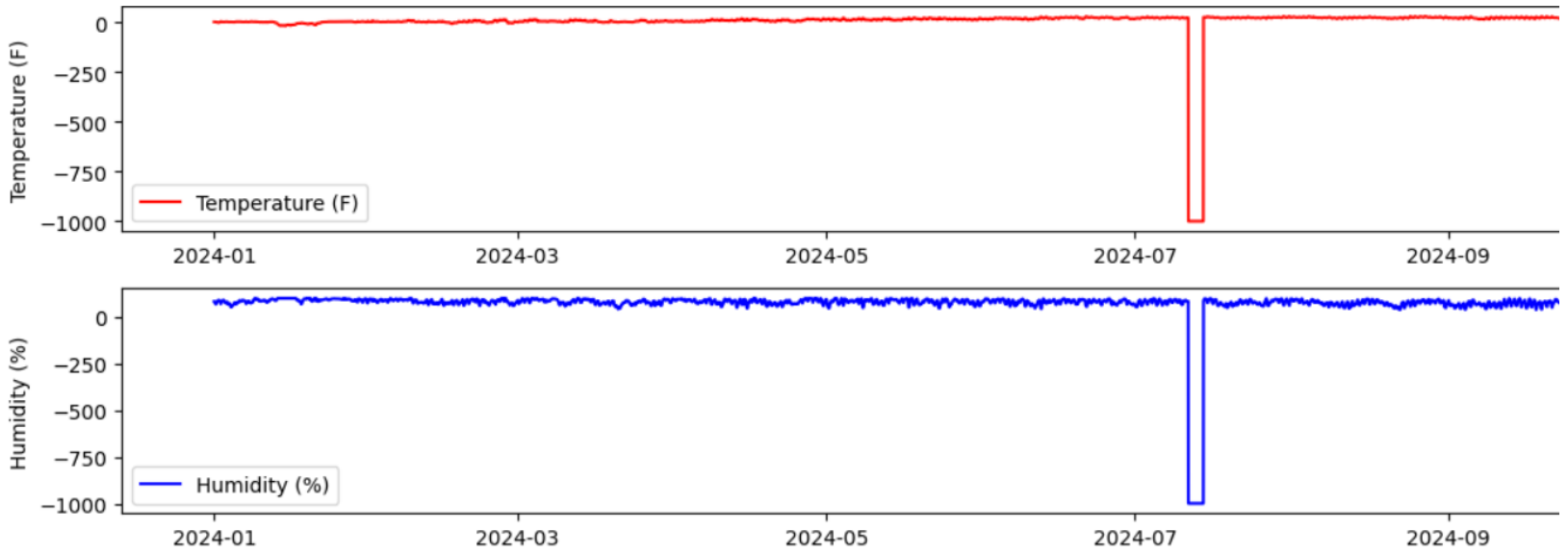


Source: https://pythonspeed.com/articles/best-file-format-for-pandas/

Examples of things to do ☺

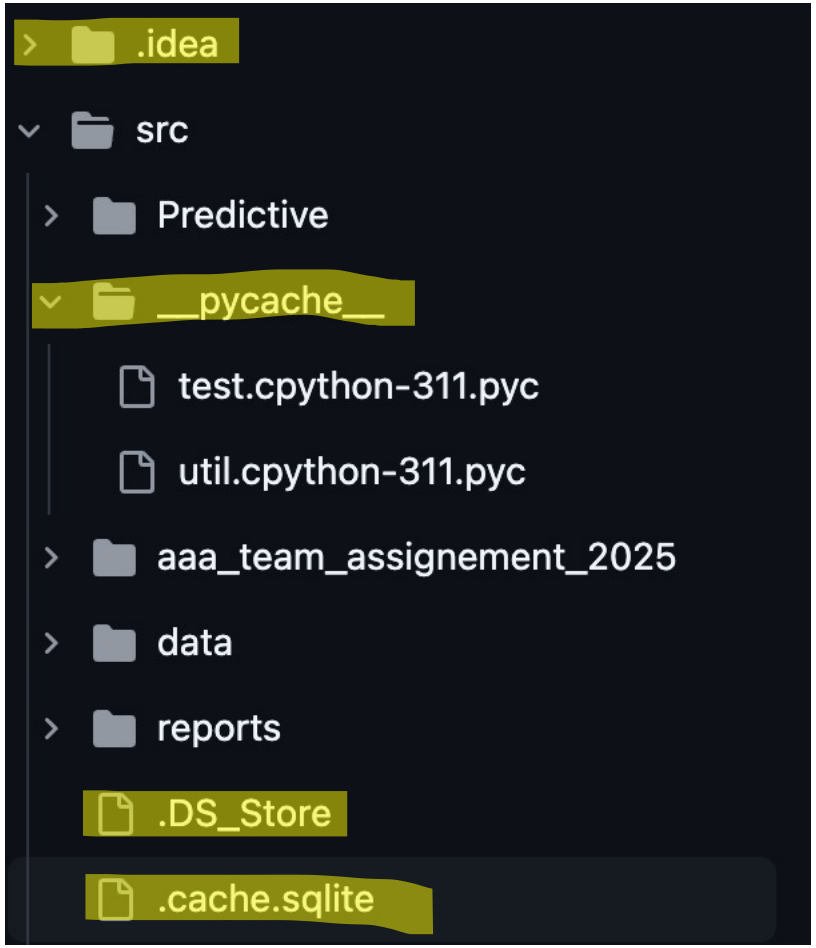
First things first, have a look at the data to make out periods where data might be corrupt!

This helps upfront with a lot of potential issues downstream.



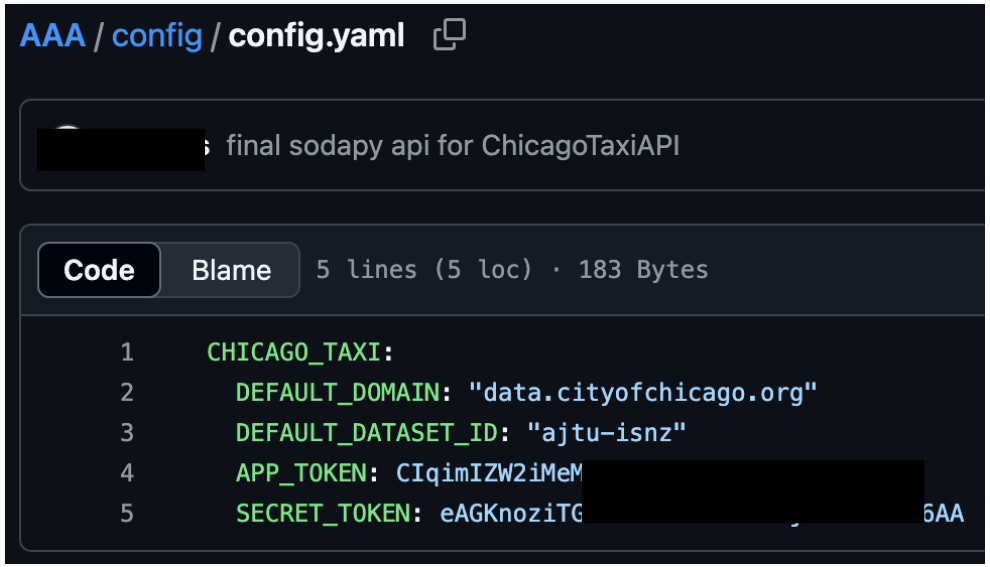
Examples of things to be aware of!

- Utilize ignore rules (a .gitignore file) to prevent pushing large, unnecessary files to the remote repository, as this adds a) an overhead on size and b) makes it unclear what is important and what is not



Things to not do! ☹

Of course, this is a private repo and this is an irrelevant uni project, but I want to make you aware that you shouldn't make a habit of this! LLMs that were trained on GitHub exposed secrets through auto-completion!



Source: https://blog.gitguardian.com/yes-github-copilot-can-leak-secrets/

Contact

For general questions and enquiries on research, teaching, job openings and new projects refer to our website at www.is3.uni-koeln.de

For specific enquiries regarding this course contact us by sending an email to the IS3 teaching address at is3-teaching@wiso.uni-koeln.de

To help us process your request efficiently, please use the following subject line format:

[AAA] <Your request subject>